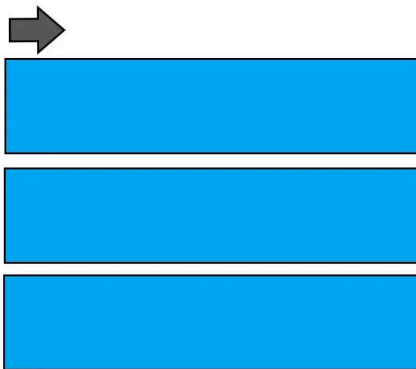




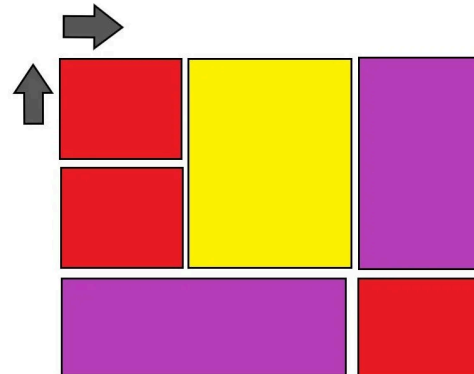
Day-6: CSS Grid

Introduction: Flex Vs Grid

Mention somewhere in benefits of Grid, that it is largely standardized across browsers



Flexbox is one-dimensional



CSS Grid is two-dimensional

Basic Grid :

```
<style>
  #container {
    display: grid;
  }
```

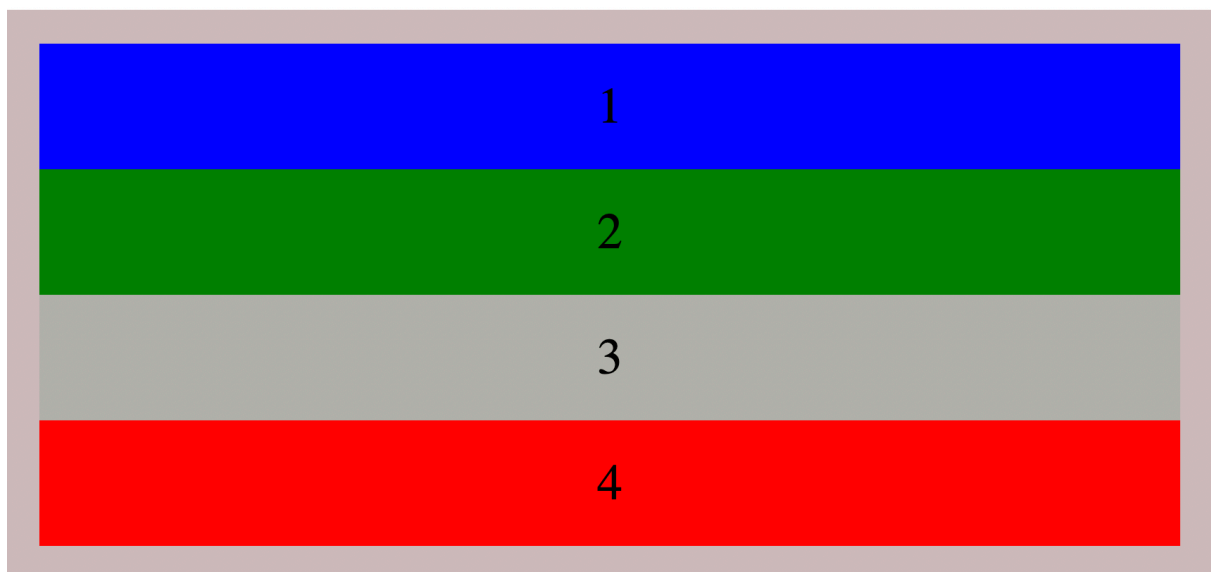
```

    </style>
  </head>
  <body>
    <div id="container">
      <div>1</div>
      <div>2</div>
      <div>3</div>
      <div>4</div>
    </div>
  </body>

```

Note : some styles are hidden like background color etc

Output:



Live code : [Codepen](#)

grid-template-columns

- Defines the columns and rows of the grid with a space-separated list of values.

```

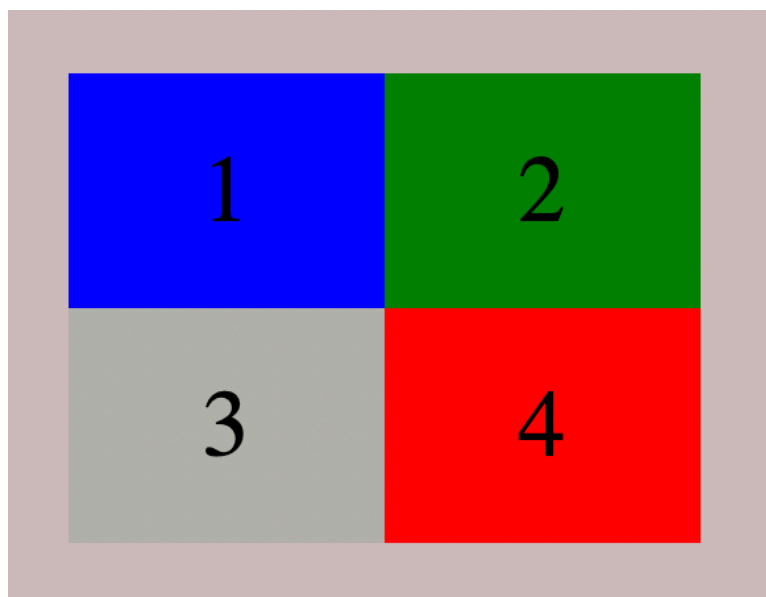
<style>
  #container {
    display: grid;
    grid-template-columns: 100px 100px; // 100px repres

```

ents column size

```
    }  
  </style>  
</head>  
<body>  
  <div id="container">  
    <div>1</div>  
    <div>2</div>  
    <div>3</div>  
    <div>4</div>  
  </div>  
</body>
```

Note : some styles are hidden like background color etc



Live code : [Codepen](#)

grid-template-rows

- We can also specify height of each row by using `grid-template-rows` property

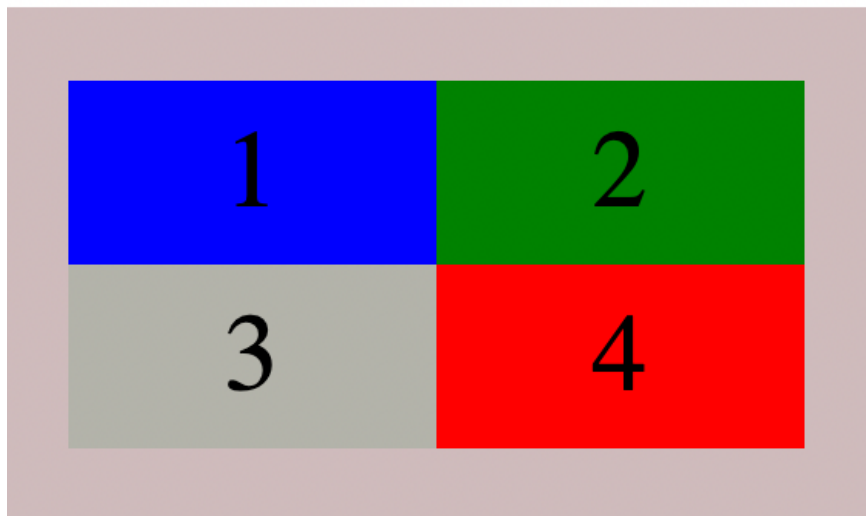
```
<style>  
  #container {  
    display: grid;  
    grid-template-columns: 100px 100px; // 100px repres
```

ents column size

`grid-template-rows: 50px 50px ; // 100px represents row height`

```
}  
</style>  
</head>  
<body>  
  <div id="container">  
    <div>1</div>  
    <div>2</div>  
    <div>3</div>  
    <div>4</div>  
  </div>  
</body>
```

Note : some styles are hidden like background color etc



Live code : [Codepen](#)

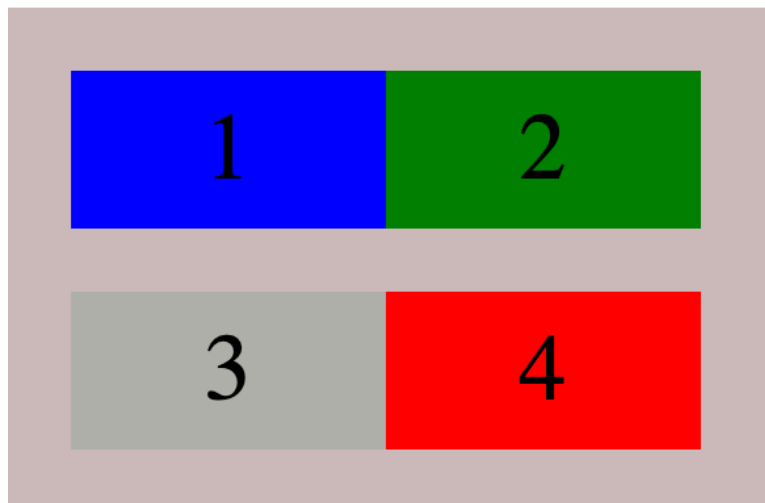
- To get gap between each item, we can use `grid-row-gap`

```
<style>  
  #container {  
    display: grid;  
    grid-template-columns: 100px 100px;  
    grid-template-rows: 50px 50px;
```

```

        grid-row-gap:20px
    }
</style>
</head>
<body>
    <div id="container">
        <div>1</div>
        <div>2</div>
        <div>3</div>
        <div>4</div>
    </div>
</body>

```



Live code : [Codepen](#)

- `grid-column-gap`

```

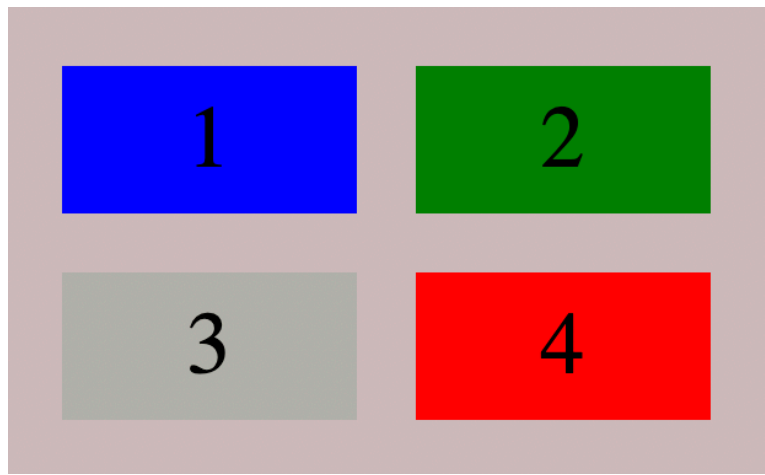
<style>
    #container {
        display: grid;
        grid-template-columns: 100px 100px;
        grid-template-rows: 50px 50px;
        grid-row-gap:20px;
        grid-column-gap:20px
    }
</style>
</head>

```

```

<body>
  <div id="container">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
  </div>
</body>

```



Live code : Codepen

- Shorthand notation for grid-gap

```

<style>
  #container {
    display: grid;
    grid-template-columns: 100px 100px;
    grid-template-rows: 50px 50px;
    /* grid-row-gap:20px;
    grid-column-gap: 20px; */
    gap:20px 20px
  }
</style>
</head>
<body>
  <div id="container">
    <div>1</div>

```

```

    <div>2</div>
    <div>3</div>
    <div>4</div>
  </div>
</body>

```

Live code : [Codepen](#)

repeat():

- The `repeat()` [CSS function](#) represents a repeated fragment of the track list, allowing a large number of columns or rows that exhibit a recurring pattern to be written in a more compact form.
- Syntax : `repeat(no_of_times,size)`
- for eg:
 - `grid-template-columns: 100px 100px` can be written as `repeat(2,100px)`
- Now the above code can be changed as

```

<style>
  #container {
    display: grid;
    grid-template-columns: repeat(2,100px);
    grid-template-rows: repeat(2,50px);
    gap:20px 20px
  }
</style>
</head>
<body>
  <div id="container">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
  </div>
</body>

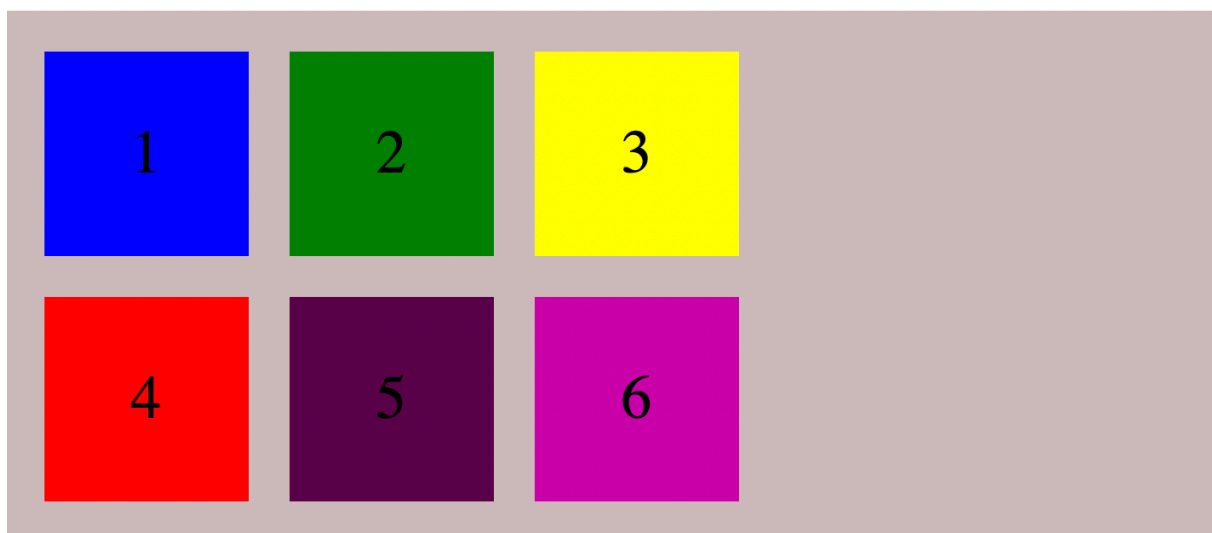
```

Live code : [Codepen](#)

- Now lets try to build a layout which has 2 rows and 3 columns with different backgrounds

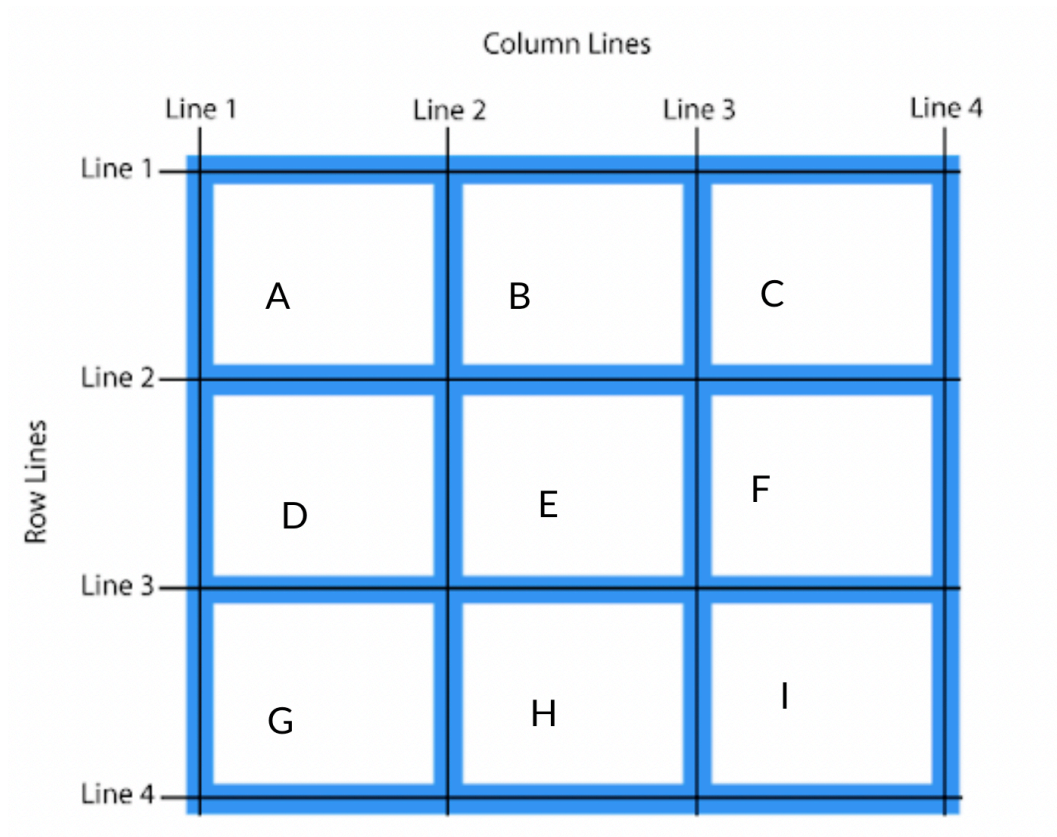
```
#container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  grid-template-rows: 100px 100px;  
  gap:20px 20px  
}  
</style>  
</head>  
<body>  
  <div id="container">  
    <div>1</div>  
    <div>2</div>  
    <div>3</div>  
    <div>4</div>  
    <div>5</div>  
    <div>6</div>  
  </div>  
</body>
```

Output:



Live code: [Codepen](#)

Line-based placement



Students Task:

- Students should know the difference between rows and columns, and identify row line
- for block, B asks these 4 values
 - row-start-line - 1
 - column-start-line - 2
 - row-end-line - 2
 - column-end-line - 3
- Also, ask for two more blocks (H and F)
- At the end of this slide, students should be able to visualize the above three values

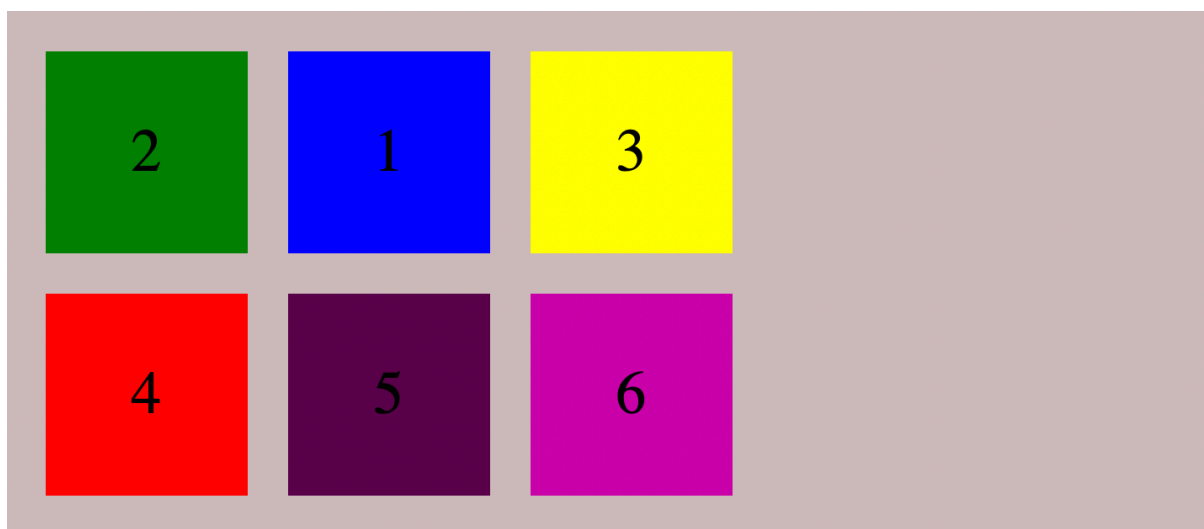

```

<style>
  #container {
    display: grid;
    grid-template-columns: 100px 100px 100px;
    grid-template-rows: 100px 100px;
    gap: 20px 20px
  }

  #container > div:nth-child(2) {
    grid-area: 1/1/2/2;
  }
</style>
</head>
<body>
  <div id="container">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
    <div>6</div>
  </div>
</body>

```

Output:



Live code: [Codepen](#)

Students Task:

- Ask students whether they can really count these lines, when this layout has large number of columns.

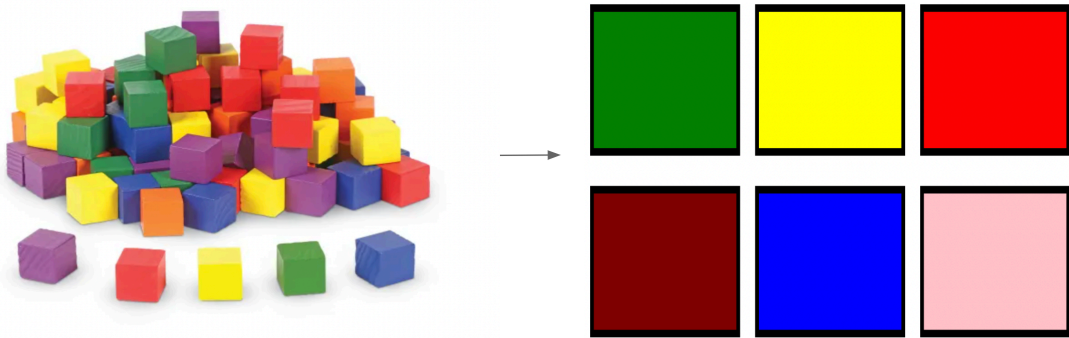
Grid-template-area method

- Since students face difficulty in line-based placement, we will now introduce them to the template-area method.
- Ask students whether they have played building blocks games in childhood

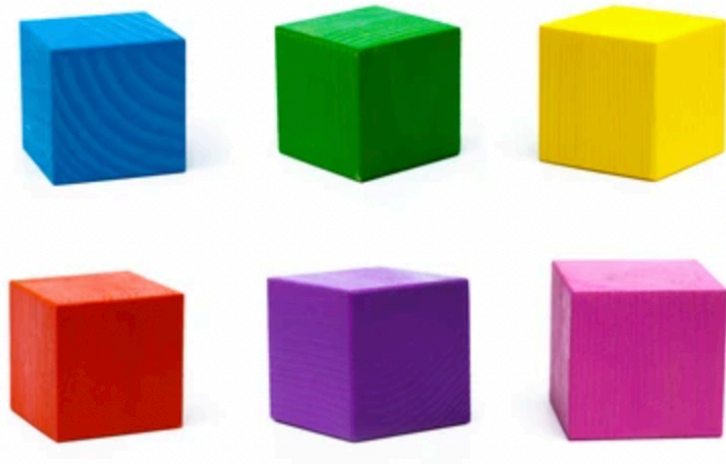


- If the answer is yes, ask them how can they build this layout using blocks

How can we achieve this?



- Answer: Just picking those colored blocks and placing it



- For building that what might be the steps student has followed
 - Identifying colors they need to pick up, based on label(color)
 - Placing those colored blocks wherever it is needed
 - In this case, we have placed boxes in this order

```
blue   green   yellow
red    maroon  pink
```

```

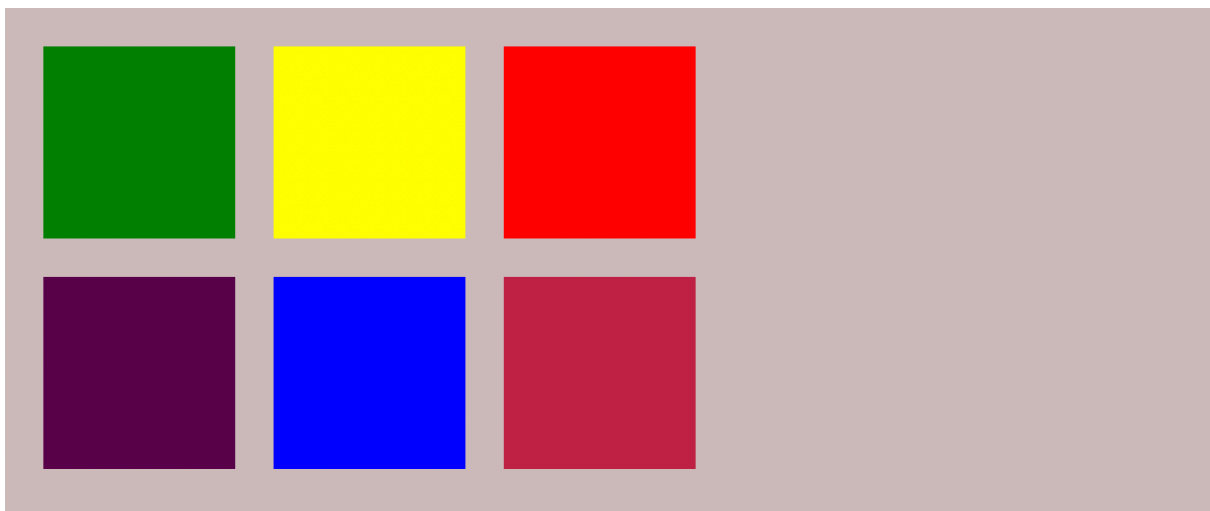
<style>
  #container {
    display: grid;
    grid-template-columns: 100px 100px 100px;
    grid-template-rows: 100px 100px 100px 100px;
    /* gap: row-gap column-gap */
    gap: 20px;
    grid-template-areas:
      "grn ylw rd"
      "mrn blu dpink"
  }

  #container > div:nth-child(1) {
    background-color: blue;
    grid-area: blu          /* label - name */
  }
  #container > div:nth-child(2) {
    background-color: green;
    grid-area: grn          /* label - name */
  }
  #container > div:nth-child(3) {
    background-color: yellow;
    grid-area: ylw
  }
  #container > div:nth-child(4) {
    background-color: red;
    grid-area: rd
  }
  #container > div:nth-child(5) {
    background-color: rgb(83, 12, 74);
    grid-area: mrn
  }
  #container > div:nth-child(6) {
    background-color: rgb(177, 52, 73);
    grid-area: dpink
  }
</style>
</head>

```

```
<body>
  <div id="container">
    <div></div>
    <div></div>
    <div></div>
    <div></div>
    <div></div>
    <div></div>
  </div>
</body>
```

Output:



- [Codepen](#)

Units of grid-template-columns:

Pixels

```
grid-template-columns: 100px 100px or repeat(2,100px)
```

Percentages

```
grid-template-columns: 50% 50% or repeat(2,50%)
```

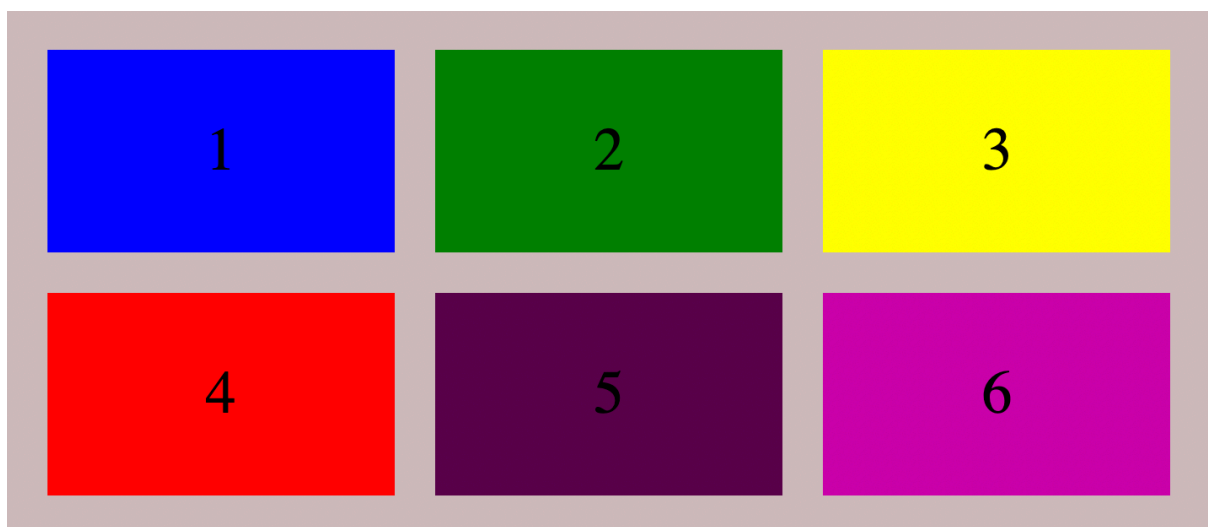
// each column will take 50% width in reference to its parent

```
grid-template-columns: 1fr 1fr or repeat(2,1fr)
```

```
// each column will take 1 fraction width in reference to  
its parent
```

```
<style>  
  #container {  
    display: grid;  
    grid-template-columns: repeat(3,1fr);  
    grid-template-rows: 100px 100px;  
    gap:20px 20px  
  }  
</style>  
</head>  
<body>  
  <div id="container">  
    <div>1</div>  
    <div>2</div>  
    <div>3</div>  
    <div>4</div>  
    <div>5</div>  
    <div>6</div>  
  </div>  
</body>
```

Output: Dividing each block into 3 fractions

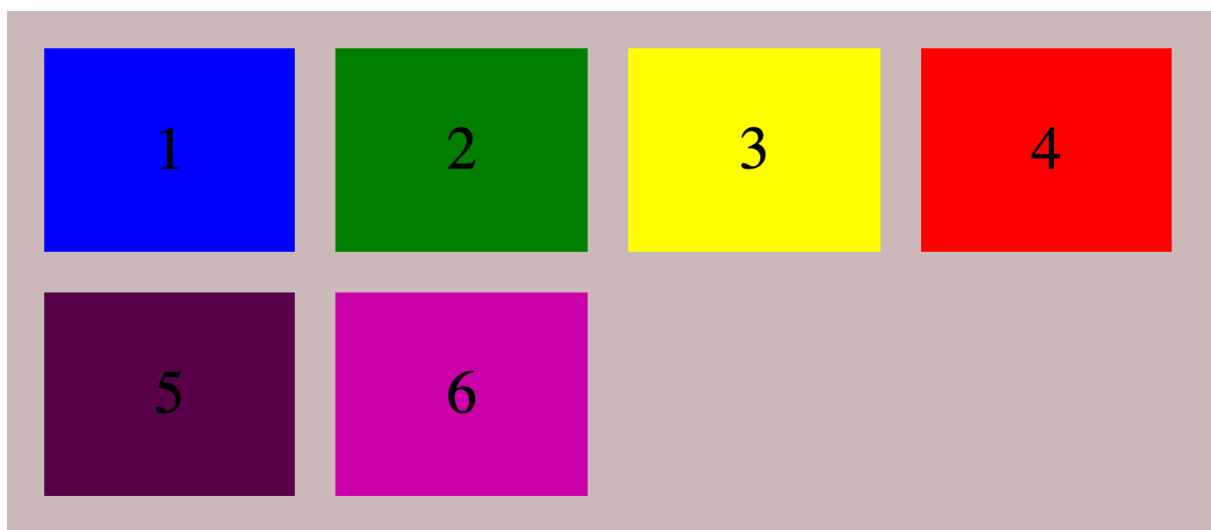


```

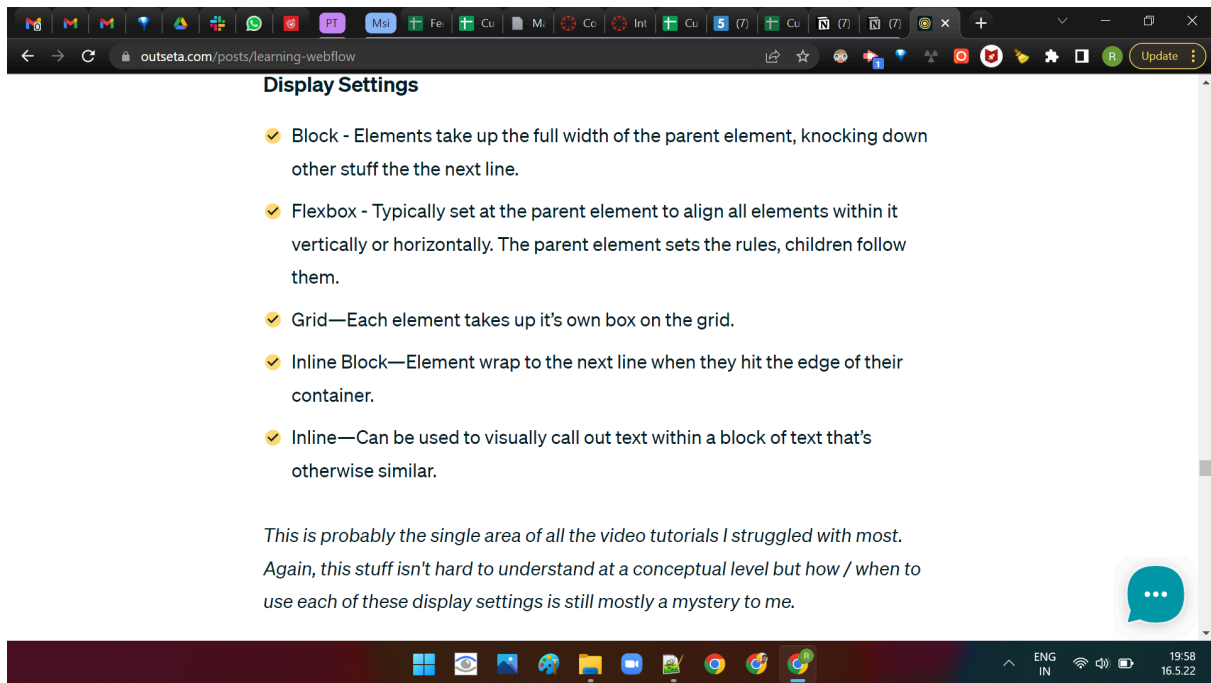
<style>
  #container {
    display: grid;
    grid-template-columns: repeat(3,1fr);
    grid-template-rows: 100px 100px;
    gap:20px 20px
  }
</style>
</head>
<body>
  <div id="container">
    <div>1</div>
    <div>2</div>
    <div>3</div>
    <div>4</div>
    <div>5</div>
    <div>6</div>
  </div>
</body>

```

Output: Dividing each block into 4 fractions



Can do a Recap of various Display settings reminding students of examples of when to use which (see the below screenshot):



WE session

Masai Calculator

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
    <style>
      h1 {
        text-align: center;
        font-size: 60px;
        color: teal;
      }

      #calculator {
        height: auto;
```

```

width: 400px;
box-shadow: rgba(100, 100, 111, 0.2) 0px 7px 29px 0
px;

margin: auto;
padding: 40px;
border-radius: 30px;
}

#display {
height: 80px;
width: 100%;
box-shadow: rgba(0, 0, 0, 0.35) 0px 5px 15px;
margin: auto;
border-radius: 10px;
}

#keyboard {
display: grid;
grid-template-columns: repeat(4, 1fr);
grid-template-rows: repeat(4, 80px);
gap: 10px;
margin-top: 20px;
}

#keyboard > div {
box-shadow: rgba(50, 50, 93, 0.25) 0px 6px 12px -2p
x,
    rgba(0, 0, 0, 0.3) 0px 3px 7px -3px;
border-radius: 10px;
font-size: 25px;
font-weight: bold;
display: flex;
justify-content: center;
align-items: center;
}

#keyboard > div:hover {
background-color: teal;

```


Keyboard CSS

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
    <script
      src="https://kit.fontawesome.com/24c494a6b6.js"
      crossorigin="anonymous"
    ></script>
    <style>
      body {
        background-color: #01bdd5;
      }
      #container {
        display: grid;
        grid-template-columns: repeat(12, 1fr);
        grid-template-rows: repeat(5, 50px);
        gap: 20px;
        width: 80%;
        margin: auto;
        background-color: #262c2c;
        padding: 40px;
        box-shadow: rgba(0, 0, 0, 0.25) 0px 14px 28px,
          rgba(0, 0, 0, 0.22) 0px 10px 10px;
        margin-top: 200px;
        border-radius: 20px;
      }

      #container > div {
        background-color: #2e3639;
```

```

    display: flex;
    justify-content: center;
    align-items: center;
    color: #11a4d1;
    font-weight: bold;
    font-size: 20px;
    box-shadow: rgba(0, 0, 0, 0.25) 0px 14px 28px,
        rgba(0, 0, 0, 0.22) 0px 10px 10px;
    border-radius: 5px;
}

#container > div:hover {
    background-color: #01bdd5;
    color: black;
    cursor: pointer;
}

#caps {
    grid-column: 1 / span 1;
}

#enter {
    grid-column: 11 / span 2;
}

#shift {
    grid-column: 11 / span 2;
}

#space {
    grid-column: 5 / span 4;
}
</style>
</head>
<body>
    <div id="container">
        <div>~</div>
        <div>1</div>

```

```
<div>2</div>
<div>3</div>
<div>4</div>
<div>5</div>
<div>6</div>
<div>7</div>
<div>8</div>
<div>9</div>
<div>0</div>
<div>delete</div>
<div id="tab">tab</div>
<div>Q</div>
<div>W</div>
<div>E</div>
<div>R</div>
<div>T</div>
<div>Y</div>
<div>U</div>
<div>I</div>
<div>O</div>
<div>P</div>
<div>\</div>
<div id="caps">caps</div>
<div>A</div>
<div>S</div>
<div>D</div>
<div>F</div>
<div>G</div>
<div>H</div>
<div>J</div>
<div>K</div>
<div>L</div>
<div id="enter">Enter</div>
<div>Shift</div>
<div>Z</div>
<div>X</div>
<div>C</div>
<div>V</div>
```

```
<div>B</div>
<div>N</div>
<div>M</div>
<div><</div>
<div>>>/div>
<div id="shift">Shift</div>
<div>
  <i class="fa-solid fa-globe"></i>
</div>
<div>control</div>
<div>option</div>
<div>alt</div>
<div id="space">Space</div>
<div>alt</div>
<div>option</div>
<div><i class="fa-solid fa-arrow-left"></i></div>
<div><i class="fa-solid fa-arrow-right"></i></div>
</div>
</body>
</html>
```